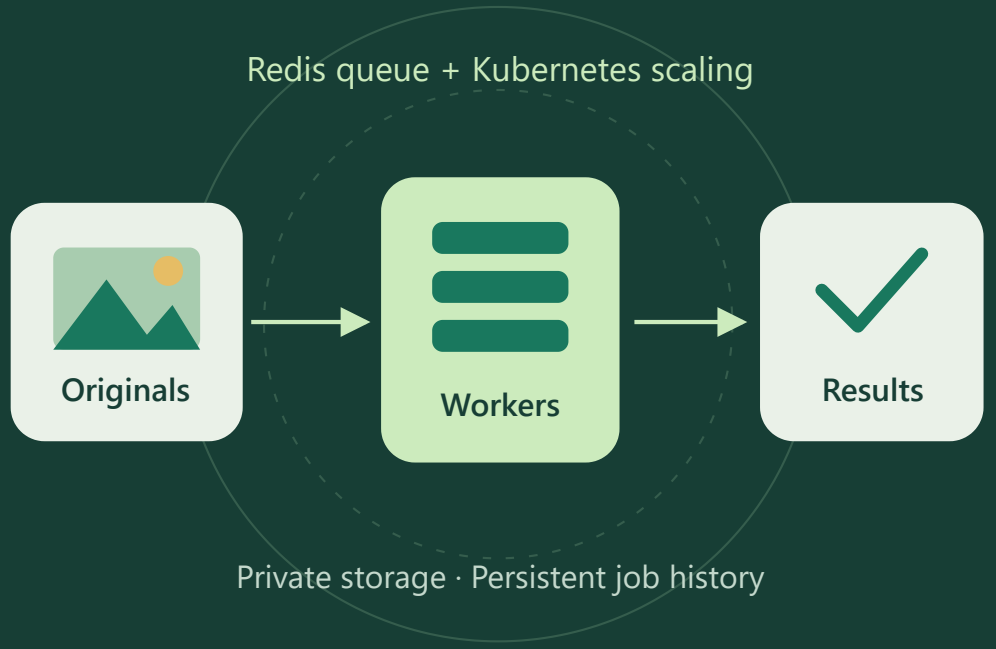
CloudFrame Image processing with queue workers.

Containerized Image Processing Service with Kubernetes Queue Workers

Presentation summary · 26 October 2026



PRODUCT
Upload, transform, preview, download

COMPUTE
Background Node + Python workers

CLOUD PATTERN
Queue-driven Kubernetes autoscaling

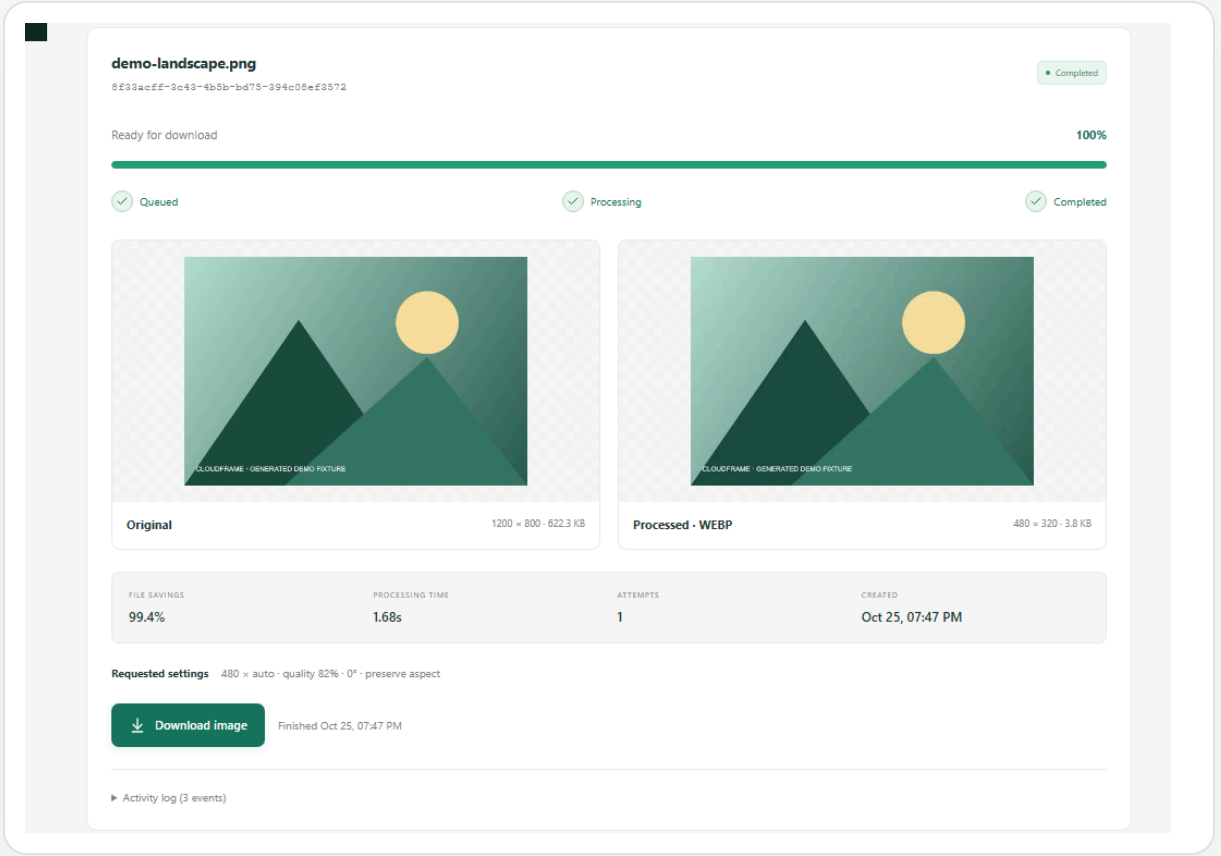
Turn uploaded images into downloadable results.

The API accepts work quickly; background workers perform the expensive image processing.

1. **Upload JPEG, PNG or WebP.**
   Select one image or a batch of up to eight.
2. **Choose the transformations.**
   Resize, quality, format, rotation, grayscale, thumbnail and text watermark.
3. **Submit and follow the job.**
   Queued → processing → completed, with persistent history and retry support.
4. **Compare and download.**
   Original/result previews and a temporary private download link.

SIX IMPLEMENTED PAGES

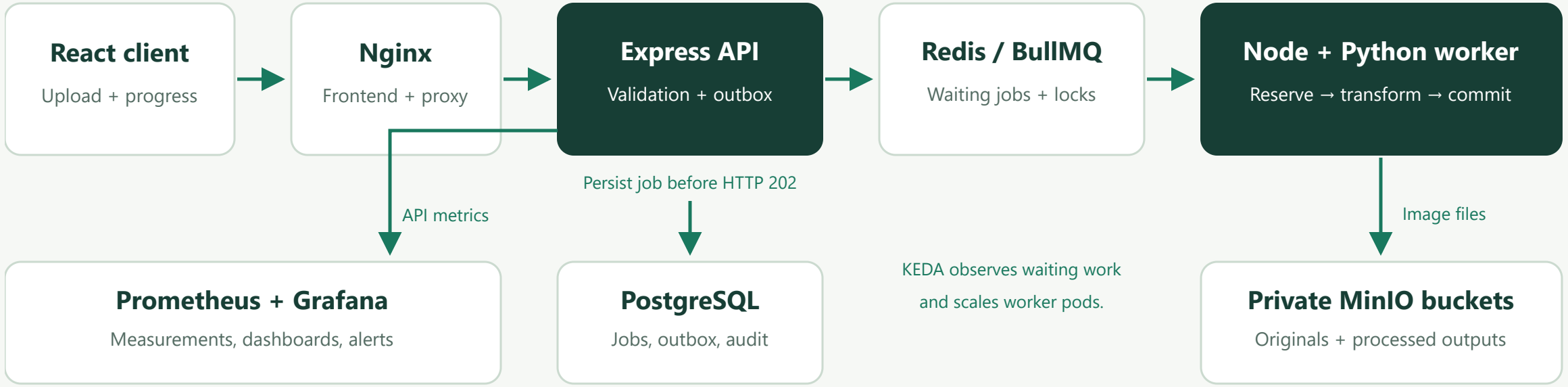
- Dashboard
- Process Images
- Job History
- Monitoring
- Architecture
- Settings



Actual Kubernetes browser-test capture · Generated demo fixture · Selected result panel

A request becomes a durable background job.

Separate services handle the interface, validation, scheduling, computation, storage and monitoring.



The API and workers share PostgreSQL job state and private MinIO storage. The UI polls job status, then requests a signed result URL.

Each tool has a specific responsibility.

LAYER	TECHNOLOGIES	ROLE IN THIS IMPLEMENTATION
Interface	React, TypeScript, Vite, Tailwind	Typed dashboard, responsive controls and production frontend build.
API	Node.js, Express, Zod, Multer	HTTP endpoints, request validation and bounded multipart uploads.
Job queue	Redis, BullMQ, Node bridge	Deliver jobs, hold worker locks, schedule retries and acknowledge work.
Image compute	Python, Pillow, boto3	Decode and transform image bytes; read/write S3-compatible objects.
Persistent data	PostgreSQL, Prisma, MinIO	Relational jobs/audits plus private original and output files.
Deployment	Docker, Compose, Nginx, Kubernetes, KEDA	Package services, route traffic, run pods and scale worker replicas.
Observability	Prometheus, Grafana, Pino	Collect metrics, visualize service health and record structured logs.

Supporting tools: Router, Axios and TanStack Query manage navigation and API state; Recharts draws activity charts; Sharp validates upload metadata. OpenAPI/Swagger document the API. Vitest, Supertest, pytest and Playwright verify behavior.

Local cluster: kind runs Kubernetes on Docker. kubectl/Kustomize deploy resources; Helm installs KEDA.

Python processes images; the Node bridge owns BullMQ consumption and database updates.

Persist first. Process in the background. Recover.

A durable PostgreSQL job row also acts as an outbox, separating job acceptance from Redis availability.

01 · UPLOAD

Validate + store

Check image type, size and dimensions. Store the original privately in MinIO.

Original preserved →

02 · ACCEPT

Commit the job

Save metadata and audit state in PostgreSQL. Return HTTP 202 and a job ID.

Durable acceptance →

03 · DISPATCH

Publish to BullMQ

The API reconciler publishes unfinished work at startup and every five seconds.

Deterministic IDs →

04 · EXECUTE

Node → Python

Node reserves the job and renews its lock. A bounded Python subprocess transforms it.

JSON stdin/stdout →

05 · COMPLETE

Commit + deliver

Upload output and commit completion before acknowledging the queue delivery.

Preview + signed URL

Temporary failures

Default three total attempts with exponential retry delays. The Python processing timeout defaults to 60 seconds.

Worker interruption

Expired reservations allow stalled work to be redelivered. Delivery is at least once; duplicate work is handled.

Private results

Unsigned objects are inaccessible. Signed download links expire after five minutes; default file retention is seven days.

Scale processing capacity from queue demand.

KEDA watches the actual BullMQ wait list and changes the Kubernetes worker Deployment replica count.

OBSERVED WORKER REPLICAS · ACTUAL RECORDED RUN

Before load	→	During backlog	→	After settling
1	→	4	→	1

24/24	23	0
Jobs completed	Peak waiting	Failed jobs

Configuration: `bull:images:wait` · target 3 waiting jobs/replica · min 1 / max 4 · polling every 5 seconds

Containers + orchestration

Docker packages services. Kubernetes manages pods, networking, readiness probes, resource limits and persistent volumes.

Asynchronous elasticity

The queue absorbs bursts. Additional combined Node/Python pods increase processing capacity while the API stays separate.

Durability + visibility

Persistent job state, outbox reconciliation, retries, metrics and audit trails make failures visible and recoverable.

Recorded verification: 25 Oct 2026. Local kind pods share the computer; this does not provision cloud VMs.

Batch size, worker count and queue depth differ.

These values describe separate parts of the system, so they do not need to match.

8

Images per batch

A fixed UI and API limit bounds a single request and its memory use. Submit another batch for more images; it is not a total project limit.

1

Connected worker

The default is one BullMQ consumer with concurrency one. Kubernetes can scale worker pods from one to four when waiting work builds up.

0

Queued work

No waiting/delayed jobs at that moment. Check active work separately. When active is also zero, the workers are usually idle.

File and pixel safeguards

Default 10 MB per image and 25 million decoded pixels. Compressed file size alone does not describe processing memory.

Different kinds of parallelism

Batch size counts images submitted together. Concurrency counts jobs per worker. Replicas count worker pods.

Reading metrics correctly

Queue depth = waiting + delayed; active is separate. Progress values are lifecycle milestones, not continuous pixel-level percentages.

Tested across API, images, browsers and containers.

Recorded implementation verification · 25 October 2026 · Results below are from executed runs.

18 passed

API / outbox / OpenAPI

Express boundaries, recovery logic and parser regressions. OpenAPI validates 13 documented paths.

16 passed

Python image processing

Actual Pillow transforms and encoders, including tests inside the deployed Python 3.12 worker image.

7 + 7 passed

Playwright browsers

Seven cases on Compose and seven on Kubernetes, including desktop/mobile and all six pages.

Real pipeline and load

Uploads, selected transformations, HTTP 202 jobs, progress, previews and signed image bytes. **Compose: 30/30 jobs. KEDA: 24/24 jobs, 1 → 4 → 1 workers.**

Recovery and infrastructure

Redis outage and worker SIGKILL recovery passed. Compose and kind ran successfully. **18 YAML/JSON files**, Kustomize, Prometheus and Grafana checks passed.

Source: docs/PROGRESS.md, final verified state and requested follow-up verification. API boundary tests mock external stores; live browser cases exercise real services. The browser suites include two explicit offline UI cases. No runtime tests were rerun solely to create this presentation.

A short demo makes the architecture visible.

Windows PowerShell · Run from the project folder with Docker Desktop using Linux containers.

```
docker compose --profile monitoring up --build -d
```

1. Open Process Images and choose a demo image.
2. Select width, output format and an effect.
3. Submit; show queued → processing → completed.
4. Compare previews and download the actual file.
5. Show Job History, Monitoring and Grafana.

Autoscaling extension: use the separate kind workload and replica watch commands in `docs/KUBERNETES.md`. Compose jobs do not enter the Kubernetes queue.

Frontend	**localhost:8080**
API / Swagger	**:3000/api/v1/docs**
MinIO console	**localhost:9001**
MinIO S3	**localhost:9000**
Prometheus	**localhost:9090**
Grafana	**localhost:3002**

Use the existing .env. If absent, create it from .env.example. Credentials are configured there; this presentation does not expose them. First-time image builds may take several minutes.

A functioning local demonstration of cloud patterns.

CloudFrame connects a usable image editor to durable asynchronous jobs, private storage, failure recovery and observed Kubernetes worker scaling.

Implemented and demonstrated

Six-page responsive application and real image transformations.

Container processing, persistent history, private previews and downloads.

Actual recovery tests, observability and KEDA scaling from one to four workers.

Current limitations and next steps

Local single-tenant academic deployment; public cloud hosting is not demonstrated.

Add authentication, ownership checks, TLS, secret management, HA and backups for public use.

Cancellation is unsupported (409). Optional Ingress and ELK were not deployed/exercised.

Presentation takeaway: the queue decouples requests from compute; Kubernetes and KEDA adjust the number of processing workers.

Further reading: README.md · docs/PROJECT_EXPLANATION.md · docs/ARCHITECTURE.md · docs/API.md · docs/KUBERNETES.md · docs/PROGRESS.md